

TORCHPAD

by Torch Wallet

Implementation Report

A Permissionless Token Launch Platform for Zilliqa 2.0

By the Torch Wallet Team

February 2026

Version 1.0

github.com/ignite-dao/torchpad

Contents

1 Executive Summary

2 Introduction and Background

The Team Behind Torchpad

The Problem Torchpad Solves

Proposal Recap

3 Platform Overview

4 Smart Contracts: The Token Factory

5 Smart Contracts: The Airdrop System

6 Smart Contracts: Bonding Curves

The Constant Product Formula

Virtual vs. Real Reserves

The Lifecycle of a Bonding Curve Token

Graduation to PlunderSwap

Fees and Price Progression

The Bonding Curve Token

7 Smart Contracts: The Fair Launch System

Lifecycle of a Fair Launch

Currencies, Caps, and Whitelists

Automatic DEX Listing and Liquidity Locking

Creator Controls

8 Security Across All Contracts

9 Testing and Quality Assurance

10 The Frontend Application

Technology Stack

Page-by-Page Walkthrough

Component Architecture and State Management

11 Backend Infrastructure

12 Ecosystem Integration

13 Milestone Delivery

14 Budget Allocation

15 The ZIL Flywheel

16 Risk Assessment and Mitigation

17 Success Metrics

18 Open Source and Licensing

19 Deployment and Operations

20 Future Considerations

21 Conclusion

A Appendix: Contract and Parameter Reference

B Appendix: Technology Stack

C Appendix: Project File Structure

1. Executive Summary

This document is the comprehensive implementation report for Torchpad, a permissionless token launch platform purpose-built for Zilliqa 2.0. Developed by the Torch Wallet team and funded through a \$50,000 grant from the gZIL Collective, Torchpad was conceived as essential infrastructure for the next phase of Zilliqa's ecosystem growth. This report details what was built, how it works, and how every deliverable maps back to the original proposal.

Torchpad delivers four distinct capabilities rolled into a single, cohesive platform. It provides a token factory for creating standard ERC-20 tokens, a bonding curve launch mechanism that uses an automated pricing algorithm, a fair launch system with full fundraising controls, and an airdrop toolkit for large-scale token distribution. All four capabilities are backed by rigorously tested smart contracts deployed on Zilliqa's EVM-compatible chain, deeply integrated with PlunderSwap (Zilliqa's primary decentralized exchange) for automatic liquidity provisioning, and wrapped in a modern web application that makes every feature accessible to non-technical users.

The original proposal allocated the budget across three areas: smart contract and backend development (60%), frontend and UI development (25%), and ecosystem integration and maintenance (15%). Every milestone has been fully delivered, and this report walks through the technical implementation in enough detail to give non-developers a thorough understanding of how the platform works and why certain design decisions were made.

2. Introduction and Background

The Team Behind Torchpad

The Torch Wallet team has been building on Zilliqa for over five years, contributing through every phase of the blockchain's evolution. Torch Wallet itself is a comprehensive, all-in-one wallet supporting both EVM and non-EVM tokens, staking and instant unstaking, Ledger hardware wallet integration, mobile and browser extension interfaces, and a built-in fiat on-ramp for purchasing ZIL directly within the wallet. The project is led by Milan Shoukri, who previously served as Head of Partnerships and Marketing at Zilliqa Research, where he contributed to the ecosystem's growth through strategy, adoption, and go-to-market execution.

This deep history with Zilliqa informed every decision in Torchpad's design. The team understands the ecosystem's strengths, its user base, and the gaps that needed to be filled to drive meaningful growth.

The Problem Torchpad Solves

Before Torchpad, launching a token on Zilliqa 2.0 required technical expertise in Solidity development, familiarity with the deployment toolchain, manual integration with decentralized exchanges for liquidity, and separate tools for distribution. There was no unified, permissionless platform where a project team could go from idea to live, tradeable token without writing a single line of code.

The original ZilSwap Launchpad during Zilliqa 1.0 demonstrated that well-designed launch tools can significantly boost ecosystem adoption and activity, helping teams raise millions and onboard users quickly. Torchpad was conceived as the next-generation successor to that concept, rebuilt from scratch for Zilliqa 2.0's EVM-compatible architecture with modern smart contract patterns and a significantly improved user experience.

Proposal Recap

The grant proposal outlined four milestones over a six-month period. The first covered the token factory and airdrop toolkit (Months 1 through 2). The second addressed fair launch contracts with DEX integration (Month 3). The third introduced bonding curves with UI re-

finements (Months 4 through 5). The fourth encompassed final testing, documentation, and public launch (Month 6). Each milestone carried a \$10,000 disbursement, with an initial \$10,000 provided upfront. This report demonstrates that all four milestones have been fully delivered.

3. Platform Overview

Torchpad allows anyone with a Zilliqa wallet to create standard ERC-20 tokens with customizable names, symbols, decimal places, and total supply. It lets them launch tokens via bonding curves, where an automated pricing algorithm handles price discovery and the token automatically graduates to PlunderSwap when it reaches a target market capitalization. It enables structured fair launches where contributors send ZIL or USDC and receive tokens proportionally after the sale ends, with optional automatic DEX listing. And it provides a mass distribution tool that can send tokens to hundreds of wallets in a single transaction.

The platform consists of three layers, exactly as proposed. The first is a suite of Solidity smart contracts deployed on Zilliqa's EVM chain that handle all on-chain logic: token creation, bonding curve trading, fair launch fundraising, liquidity management, and airdrop distribution. The second is a modern web application built with Next.js 15 and React 19 that provides an intuitive, step-by-step interface for every platform feature. The third is a lightweight Cloudflare Workers backend with D1 (SQLite) database and R2 object storage that stores token metadata, project descriptions, social links, and images. This backend exists solely to enrich the on-chain data with human-readable context; all financial logic lives exclusively on-chain.

Torchpad supports two Zilliqa EVM networks: the production mainnet (Chain ID 32769) and a testnet (Chain ID 33101) where developers can experiment without risking real funds. Users switch between networks using the network selector in the top navigation bar, and the application dynamically loads the correct smart contract addresses for the selected network.

4. Smart Contracts: The Token Factory

The `ForgeTokenFactory` contract is the simplest entry point into Torchpad. Its job is straightforward: let anyone deploy a new ERC-20 token in a single transaction. Before Torchpad, creating a token on Zilliqa required writing Solidity code, compiling it, deploying it manually, and verifying it on a block explorer. The factory reduces all of that to filling in four fields: name, symbol, decimal places, and total supply.

When a user creates a token, the factory first validates the inputs. Decimal places must be between 0 and 18 (the ERC-20 standard maximum), and the total supply must be greater than zero. These simple checks prevent deployment of tokens that would be immediately broken. If the platform has a creation fee configured (denominated in ZIL), the contract collects it from the transaction value. Any excess ZIL sent by the user is automatically refunded.

The factory then deploys a new instance of the `ForgeStandardERC20` contract. This is a clean, standard OpenZeppelin ERC-20 token with the specified parameters. The entire token supply is minted directly to the creator's wallet address in the same transaction. Finally, a `TokenCreated` event is emitted containing the new token's address, the creator's address, and all the token metadata. The frontend listens for this event to display confirmation details.

Each token created through the factory is fully autonomous after deployment. It has no mint function beyond the initial supply created in the constructor. There is no owner with special privileges, no pause mechanism, and no upgrade path. This design was deliberate: token holders never have to worry about the creator inflating the supply or freezing transfers. The factory itself has two administrative functions, both protected by OpenZeppelin's `Ownable2Step` pattern (which requires a two-step ownership transfer to prevent accidental loss of control): one to update the creation fee, and one to change the treasury address that receives those fees.

5. Smart Contracts: The Airdrop System

The ForgeAirdropper contract enables mass token distribution. Airdrops are a common mechanism in blockchain ecosystems for distributing tokens to community members, rewarding early adopters, or seeding initial token holder bases. Without a tool like this, a project would need to manually send individual transactions to each recipient, which is expensive in gas fees and time-consuming.

The airdrop process works in two steps. First, the sender must approve the ForgeAirdropper contract to spend their tokens. This is a standard ERC-20 mechanism: by calling the token's `approve` function, the sender gives the airdropper permission to transfer tokens on their behalf. Second, the sender calls one of two functions. The first, `airdrop`, accepts a token address, an array of recipient addresses, and an array of corresponding amounts, sending different amounts to different recipients. The second, `airdropEqual`, accepts a token address, an array of recipients, and a single amount, distributing the same allocation to everyone.

The contract loops through each recipient and calls `safeTransferFrom` to move tokens from the sender to each address. The "safe" prefix means the transfer will revert the entire transaction if any individual transfer fails, ensuring atomicity: either every recipient gets their tokens, or nobody does. This is critical for trust. A partial airdrop with some transfers silently failing would create confusion and disputes.

The frontend enforces a maximum of 500 recipients per transaction. This limit is practical rather than technical. The smart contract itself has no hardcoded limit, but larger batches consume more gas and may exceed block gas limits. Five hundred recipients strikes a balance between efficiency and reliability. Projects with larger distribution lists can simply perform multiple airdrop transactions. Like the token factory, the airdropper has a configurable platform fee collected in native ZIL, charged per-transaction rather than per-recipient to keep costs predictable regardless of distribution size.

6. Smart Contracts: Bonding Curves

The bonding curve system is Torchpad's most sophisticated feature. It provides an automated way to launch tokens with instant liquidity and mathematically determined pricing, similar to the mechanism that gained popularity through platforms like pump.fun on Solana. Three contracts work together to make this possible: the ForgeBondingCurveFactory, the ForgeBondingCurvePool, and the ForgeBondingCurveToken.

A bonding curve is a mathematical function that defines the price of a token based on its circulating supply. In Torchpad's implementation, the price increases as more tokens are purchased and decreases as tokens are sold back. There is no order book and no counterparty needed. The smart contract itself acts as both buyer and seller, using a formula to determine prices in real time. This approach solves the fundamental chicken-and-egg problem of token launches: new tokens typically have no liquidity and no price discovery mechanism. A bonding curve provides both from the very moment the token is created, allowing anyone to buy in at any time at a fair, algorithmically determined price.

The Constant Product Formula

Torchpad uses a constant product formula for its bonding curve, the same mathematical model that powers Uniswap and most automated market makers in the DeFi space. The idea is simple: the contract maintains two "virtual" reserves, one for tokens and one for ZIL, and their product must always remain constant.

$$\text{virtualTokenReserve} \times \text{virtualZilReserve} = k \quad (\text{constant})$$

When someone buys tokens, ZIL flows into the virtual ZIL reserve, the virtual token reserve decreases, and the constant k is maintained. This means each successive purchase pushes the price higher. Selling reverses the process. The price at any given moment is simply the ratio of the two reserves: `price = virtualZilReserve / virtualTokenReserve`.

Virtual vs. Real Reserves

A critical distinction in Torchpad's bonding curve is between virtual reserves and real reserves. Virtual reserves exist only for price calculations. They include initial seed values that create the starting price curve: 22,500 ZIL on the ZIL side and the full supply of one billion tokens on the token side. Nobody actually deposited that initial 22,500 ZIL. It's a mathematical starting point that establishes where the price curve begins.

Real reserves, by contrast, track the actual ZIL deposited by buyers. Only real ZIL can be withdrawn by sellers or used during graduation. This separation ensures that the initial virtual ZIL reserve (which no one actually deposited) can never be extracted. Sellers can only withdraw ZIL that was genuinely deposited by other participants, preventing any phantom liquidity drain.

The Lifecycle of a Bonding Curve Token

Every bonding curve token goes through two distinct phases. During the **Trading** phase, the token is live on its bonding curve. Anyone can buy tokens by sending ZIL to the pool contract, and token holders can sell back to the pool at the current price. During this phase, the token cannot be transferred between wallets. It can only be bought from or sold to the pool. This restriction prevents early buyers from creating secondary markets that could undermine the bonding curve's price discovery mechanism.

The second phase is **Graduation**. When the token's market capitalization reaches the graduation threshold (8 million ZIL by default), the pool automatically transitions. The remaining real ZIL and unsold tokens are deposited as liquidity on PlunderSwap V3, Zilliqa's primary decentralized exchange. The liquidity position NFT is permanently burned (sent to an unrecoverable address), ensuring the liquidity can never be removed by anyone, including the Torchpad team. At this point, the token's transfer restriction is lifted, and it trades freely on PlunderSwap like any other token.

Graduation to PlunderSwap

Graduation is the most important moment in a bonding curve token's lifecycle. Here is exactly what happens when the market cap threshold is reached.

First, a graduation fee (2.5% by default) is deducted from the real ZIL reserves and sent to the platform treasury. The remaining real ZIL is then wrapped into WZIL (Wrapped ZIL), which is required for interaction with PlunderSwap's smart contracts. A new liquidity pool is created on PlunderSwap V3 with the token/WZIL pair, and the initial price is set to match the bonding curve's current price. A full-range liquidity position is minted, providing liquidity across the entire price spectrum. The V3 liquidity position NFT is then sent to address `0xdead`, an unrecoverable address. This permanently locks the liquidity. Finally, the pool calls the token's `enableTrading` function, which removes the transfer restriction, allowing the token to trade freely.

Fees and Price Progression

Every buy transaction incurs a 1% trading fee (configurable by the platform). This fee is collected in ZIL before the remaining amount is used to calculate the token output. Sell transactions incur the same fee, deducted from the ZIL proceeds before the seller receives their funds.

The price progression follows a predictable curve. Starting from a very low initial price of approximately 0.0000225 ZIL per token (with default parameters), it rises smoothly as more tokens are purchased. By the time the pool reaches its graduation market cap of 8 million ZIL, the price has risen to approximately 2.844 ZIL per token. The total real ZIL collected at graduation is approximately equal to the square root of the initial virtual reserve multiplied by the graduation market cap, minus the initial virtual reserve.

The factory allows the platform owner to configure all these parameters for newly created pools:

Parameter	Default Value	Description
Graduation Market Cap	8,000,000 ZIL	Market cap at which the token graduates to PlunderSwap
Initial Virtual ZIL Reserve	22,500 ZIL	Starting virtual reserve; controls the initial price and curve shape
Trading Fee	1% (100 basis points)	Fee on every buy and sell transaction
Graduation Fee	2.5% (250 basis points)	One-time fee deducted during graduation
V3 Fee Tier	1% (10000)	PlunderSwap V3 pool fee tier for the graduated liquidity
Creation Fee	Configurable	Flat ZIL fee to create a new bonding curve pool

The Bonding Curve Token

The bonding curve token is a special variant of the standard `ForgeStandardERC20`. It adds a transfer lock controlled by the pool contract. During the trading phase, the token's internal transfer function checks whether trading has been enabled. If not, it only allows transfers involving the pool contract itself (for buys and sells) and mint/burn operations. All peer-to-peer transfers are blocked.

This lock is a critical safety feature. Without it, early buyers could send tokens to other wallets and create secondary markets, potentially manipulating the bonding curve price or front-running graduation. The lock ensures orderly operation until the token is ready for open trading. Once the pool calls `enableTrading`, the restriction is permanently removed and the token behaves like any standard ERC-20.

7. Smart Contracts: The Fair Launch System

The fair launch system provides a more traditional fundraising mechanism. Unlike bonding curves, which provide continuous price discovery, fair launches operate as structured fundraising events with defined start and end times, contribution limits, and proportional token distribution. Two contracts power this system: the `ForgeFairLaunchFactory` and the `ForgeFairLaunchPool`.

Lifecycle of a Fair Launch

A fair launch begins when a project creator deploys a pool through the factory, specifying the token address, the number of tokens for sale, the fundraising currency (ZIL or USDC), soft and hard caps, maximum per-wallet contribution limits, start and end times, and liquidity preferences. After creation but before the start time, the pool sits in an **Upcoming** state. The creator can configure whitelists during this window.

Once the start time arrives, the pool goes **Live**. Contributors send ZIL or USDC, and each contribution is tracked against both the per-wallet maximum and the overall hard cap. When the end time passes, the contract evaluates whether the soft cap was met. If it was, the sale is successful and the creator can finalize it. If it fell short, the sale has failed and all contributors can claim full refunds.

During **Finalization**, the creator calls a function that optionally creates liquidity on PlunderSwap and calculates the creator's proceeds. After finalization, contributors claim their allocated tokens. The allocation is proportional: if someone contributed 10% of the total raised, they receive 10% of the tokens for sale.

Currencies, Caps, and Whitelists

Fair launches support two fundraising currencies. ZIL is simpler for users and drives direct demand for Zilliqa's native currency. USDC protects both the project and contributors from ZIL price volatility during the fundraising period.

The dual-cap system protects both sides of a launch. The soft cap is the minimum amount that must be raised for the sale to succeed; if it's not met, all contributions are refundable. The hard cap is the maximum the sale will accept; once reached, no more contributions are allowed. If no hard cap is set (set to zero), the sale has no upper limit.

Fair launches can optionally enable a whitelist using Merkle trees. A Merkle tree is a data structure that allows efficient proof of membership. The creator provides a root hash of the approved addresses, and each whitelisted contributor provides a Merkle proof when contributing. The smart contract verifies the proof on-chain, ensuring only approved addresses can participate. The creator can update the whitelist root at any time and can toggle enforcement on or off, allowing for a whitelisted pre-sale that opens to the public mid-sale.

Automatic DEX Listing and Liquidity Locking

One of the fair launch system's most powerful features is automatic liquidity listing. When auto-listing is enabled, the finalize function automatically creates a trading pair on PlunderSwap using a portion of the raised funds and reserved tokens.

The creator sets a liquidity percentage between 51% and 100%, which determines how many additional tokens are reserved for liquidity. For example, if 1,000,000 tokens are for sale and the liquidity percentage is 60%, then 600,000 additional tokens are set aside. These extra tokens are transferred to the pool at creation time, ensuring they are locked and available when needed. The system supports both PlunderSwap V2 (classic AMM model) and V3 (concentrated liquidity), with V3 supporting four fee tiers: 0.01%, 0.05%, 0.25%, and 1%.

To protect token buyers from liquidity rug-pulls, fair launches support liquidity locking with six duration options: no lock, 30 days, 90 days, 180 days, 365 days, or indefinite (permanent lock). When the lock period expires, the creator can transfer the LP tokens (V2) or the liquidity position NFT (V3) to their own wallet.

Creator Controls

Fair launch creators have several administrative capabilities during the lifecycle of their sale. They can pause and unpause contributions without cancelling the sale entirely, which is useful if issues arise during a live sale. They can cancel the sale before it starts and recover all tokens. They can update the Merkle whitelist root or toggle enforcement at any time.

After finalization, they can withdraw the raised funds minus any amount used for DEX liquidity. And once all contributors have claimed their tokens, they can sweep any remaining token dust left in the pool due to rounding.

8. Security Across All Contracts

Security was a primary consideration throughout Torchpad's smart contract development. The same patterns are consistently applied across every contract in the system.

Every function that transfers value uses OpenZeppelin's **ReentrancyGuard**, which prevents malicious contracts from recursively calling back into the function during execution. This is the industry-standard defense against one of the most common smart contract attack vectors.

All ERC-20 token transfers use OpenZeppelin's **SafeERC20** library, which handles tokens that don't return boolean values on transfer (a common deviation from the standard) and reverts on failure rather than silently succeeding.

All factory contracts use **Ownable2Step** instead of the simpler Ownable pattern. This requires the new owner to actively accept ownership, preventing accidental transfer to an incorrect or inaccessible address. Every user-facing function validates its inputs before executing: token names and symbols must not be empty, supply must be positive, fees must be within reasonable bounds (maximum 10%), and addresses must not be zero.

Trading functions in the bonding curve accept a **minimum output parameter**, allowing users to specify the least amount they are willing to receive. If market conditions change between the user's transaction submission and execution, the transaction reverts rather than executing at an unfavorable price. This is standard slippage protection.

All fee-collecting functions automatically **refund any excess native currency** sent above the required fee, preventing users from accidentally overpaying. And across the entire codebase, the contracts are written in Solidity 0.8.24, which has built-in overflow and underflow protection, eliminating an entire class of arithmetic vulnerabilities that plagued earlier Solidity versions.

9. Testing and Quality Assurance

All smart contracts include comprehensive test suites written using Foundry's testing framework. The token factory tests cover default and custom decimal token creation, fee collection and refunding, input validation for zero supply and excessive decimals, ownership functions, and the withdrawal mechanism.

The airdropper tests verify array length mismatch detection, successful variable-amount and equal-amount airdrops, insufficient allowance handling, and fee processing. The bonding curve tests cover pool creation, buy and sell operations, price progression, graduation mechanics, fee collection, minimum trade amount enforcement, and the PlunderSwap V3 integration using mock contracts for the WETH9 wrapper and the V3 position manager. The fair launch tests verify token transfer during launch creation, native ZIL contribution flows, finalization, and claim distribution.

Mock contracts are used for external dependencies to ensure the tests are self-contained and deterministic. The MockWETH contract provides a full WETH9-compatible implementation with deposit, withdraw, and transfer functions. The MockPositionManager simulates the V3 non-fungible position manager with its MintParams structure and mint function. This approach lets the test suite run in isolation without depending on the state of any external protocol.

10. The Frontend Application

The frontend is the face of Torchpad. It transforms complex blockchain operations into guided, step-by-step workflows that anyone can follow. This component represents the 25% (\$12,500) frontend allocation from the grant budget.

Technology Stack

The application is built with **Next.js 15** using the App Router architecture, which enables React Server Components for faster initial page loads. All interactive components use **React 19**. Styling relies on **Tailwind CSS v4** with a custom color theme, paired with **shadcn/ui** components built on Radix UI primitives for accessible, customizable interface elements like buttons, cards, dialogs, forms, and tables.

Wallet connection and smart contract interaction are handled by **Wagmi** and **RainbowKit**, industry-standard libraries in the Web3 space. RainbowKit provides the wallet connection modal supporting Torch Wallet, MetaMask, and WalletConnect. Wagmi provides React hooks for reading from and writing to smart contracts. Complex forms use **React Hook Form** with **Zod** schemas for type-safe validation, and data fetching is managed through **TanStack React Query** for automatic caching and background refetching.

Page-by-Page Walkthrough

Landing Page. The landing page serves as both the front door and the primary discovery interface. It features an animated hero section with a gradient mesh background, floating blobs, and a subtle grid scroll animation. Below the hero, the page displays a mixed feed of active launches from both bonding curve and fair launch factories. Users can filter by type and status. Each launch appears as a card showing key metrics: current price, market cap, and graduation progress for bonding curves; amount raised, cap progress, and time remaining for fair launches. Prominent action cards guide users toward the three main creation flows.

Create Token Page. Deliberately simple. Four form fields (name, symbol, decimals, total supply) and a create button. The current creation fee is displayed dynamically by reading from the smart contract. After submission, the page parses the `TokenCreated` event from

the transaction receipt and displays the new token's address with a block explorer link and a copy button.

Airdrop Page. A two-step flow for batch distribution. Users enter a token address, which triggers automatic detection of the token's symbol and decimal places. They paste a CSV-formatted list of recipients and amounts. The interface validates all entries, highlights errors, and shows a preview table with statistics including total recipients, total amount, and current allowance. If the airdropper contract doesn't have sufficient allowance, an Approve button grants permission. Once approved, the Airdrop button triggers the distribution transaction.

Bonding Curve Launch Page. Collects the token name, symbol, project description, social links, and an optional project image (drag-and-drop, 2MB limit). The launch is a two-step operation: first the smart contract transaction creates the pool on-chain, then the metadata and image are uploaded to Cloudflare storage through the backend API. The user is redirected to their new token's trading page after both steps complete.

Bonding Curve Trading Page. Each bonding curve token has a dedicated page at `/discover/[address]`. It features a dual-tab trading panel with Buy (ZIL to tokens) and Sell (tokens to ZIL) interfaces, real-time price quotes that update as the user types, a progress bar showing the journey toward graduation, detailed statistics, project metadata with social links, and a PlunderSwap link after graduation. The trading panel includes slippage protection: the minimum output is set to 99% of the quoted amount, providing a 1% buffer against price movement.

Fair Launch Creation Page. The most complex form in the application, organized as a four-step wizard. Step 1 verifies the token address and auto-detects metadata. Step 2 configures all fundraising parameters: tokens for sale, currency, caps, contribution limits, timing, liquidity percentage, auto-listing preferences, lock duration, and whitelist settings. Step 3 collects project details and a logo image. Step 4 presents a comprehensive review with a checklist showing approval status before the final launch transaction.

Fair Launch Detail Page. Adapts to the launch's current state. When upcoming, it shows a countdown and sale parameters. When live, it displays a contribution form with progress bar and the user's current contribution. When awaiting finalization, it shows a finalize button for the creator. When finalized, it provides a claim button for contributors. When failed, it offers refund functionality. When cancelled, it displays cancellation status and refund options.

Discovery and Token List Pages. The discovery page provides a unified browsing interface for all active launches across both bonding curves and fair launches. The token list page maintains a searchable registry of all tokens created through the factory, displaying name, symbol, decimals, supply, addresses, and transaction hashes.

FAQ, Terms, and Privacy Pages. The FAQ is organized into five categories with expandable accordions. The Terms of Service include risk warnings, eligibility requirements, fee disclosures, and liability limitations. The Privacy Policy covers data collection, blockchain data immutability, third-party integrations, and cookie usage.

Component Architecture and State Management

The frontend is organized into several layers. The **App Shell** combines the sidebar navigation, top bar, and content area into a cohesive interface with a maximum width of 64rem. The **Navigation Sidebar** is collapsible with sections for navigation, tools, and resources, including a promotional Torch Wallet banner. The **Top Bar** features the logo, sidebar toggle, beta badge, theme toggle, network selector, and wallet connection button with a back-drop blur effect.

State management uses a layered approach. A **Network Context** tracks the selected chain, persists the selection to `localStorage`, respects URL query parameters, and falls back to the connected wallet's chain. The **Web3 Provider Stack** combines Wagmi for blockchain connection, React Query for data fetching, and RainbowKit for wallet UI. All blockchain reads use Wagmi's `useReadContract` hook with automatic refetching, and writes use `useWriteContract` with transaction receipt confirmation.

11. Backend Infrastructure

The backend layer is intentionally lightweight. It exists solely to store and serve metadata that enriches the on-chain experience. All financial logic, all token operations, all liquidity management happens exclusively on-chain through the smart contracts.

The backend runs on **Cloudflare Workers**, a serverless compute platform that executes code at the network edge, close to users worldwide. This provides sub-100ms response times regardless of the user's location. The Next.js application is deployed to Cloudflare using the OpenNext framework, which adapts Next.js's server-side features for the Cloudflare Workers runtime.

Token and launch metadata is stored in **Cloudflare D1**, a serverless SQLite database. The schema uses a single `token_metadata` table with fields for the pool address (primary key), chain ID, launch type, name, symbol, description, image URL, website, Twitter handle, Telegram link, and creation timestamp. The table is indexed on chain ID, launch type, and creation date for efficient querying.

Project images are stored in **Cloudflare R2**, an S3-compatible object storage service. Images are stored with structured key formats and served with aggressive caching headers (one year, immutable) since each image is uniquely keyed to its pool address.

The backend exposes four API endpoints. A POST endpoint at `/api/launches/metadata` accepts multipart form data with pool details and an optional image. A GET endpoint at the same path returns all metadata entries for a given chain, optionally filtered by launch type. A GET endpoint at `/api/launches/metadata/[address]` returns metadata for a single pool. And an image-serving endpoint at `/api/launches/images/[...path]` streams images from R2 with appropriate content-type headers.

12. Ecosystem Integration

Deep integration with the Zilliqa 2.0 ecosystem was a core focus of the proposal, backed by the 15% (\$7,500) ecosystem allocation.

PlunderSwap. Torchpad is tightly integrated with PlunderSwap, Zilliqa's primary decentralized exchange. Bonding curve graduation automatically creates liquidity pairs on PlunderSwap V3 with permanently locked liquidity. Fair launches can auto-list on either PlunderSwap V2 or V3 upon finalization, with configurable fee tiers and liquidity lock durations. The integration uses PlunderSwap's official contract addresses for the V2 router and factory, V3 factory, pool deployer, migrator, and non-fungible position manager.

Torch Wallet. Torchpad is designed to work seamlessly with Torch Wallet through RainbowKit's wallet connection interface. The sidebar includes a Torch Wallet promotional banner, and the platform's visual design language is consistent with Torch Wallet's branding.

Block Explorer. Every address, transaction hash, and token displayed in Torchpad includes a direct link to Zilliqa's Otterscan block explorer, allowing users to independently verify every on-chain action.

Contract Verification. All deployed smart contracts are verified on Sourcify, an open-source contract verification service. The project includes a dedicated shell script that automates the verification process using Foundry's built-in verification command. Verified contracts allow anyone to read the source code directly on block explorers, promoting transparency and trust.

13. Milestone Delivery

This section maps every deliverable to the four milestones defined in the original proposal.

Milestone 1: Token Factory and Airdrop Toolkit (Months 1-2, \$10,000)

The token factory contract (ForgeTokenFactory.sol and ForgeStandardERC20.sol) was deployed, tested, and verified. It creates standard ERC-20 tokens with configurable parameters. The airdrop contract (ForgeAirdropper.sol) was deployed, tested, and verified, supporting both variable-amount and equal-amount distributions to up to 500 recipients. The frontend includes complete token creation (/create-token) and airdrop (/airdrop) workflows with validation, fee display, CSV-based recipient entry, and token approval management. The backend for launch metadata was built out with the Cloudflare D1 database schema, API endpoints for storage and retrieval, and R2 storage for images.

Milestone 2: Fair Launch with DEX Integration (Month 3, \$10,000)

The fair launch contracts (ForgeFairLaunchFactory.sol and ForgeFairLaunchPool.sol) were deployed, tested, and verified. They support ZIL and USDC fundraising, Merkle whitelist verification, hard/soft caps, and proportional distribution. The frontend includes a four-step creation wizard and dynamic detail pages that adapt to each launch's state. The liquidity locking mechanism supports six lock durations for both V2 LP tokens and V3 position NFTs.

Milestone 3: Bonding Curves with UI Refinements (Months 4-5, \$10,000)

The bonding curve contracts (ForgeBondingCurveFactory.sol, ForgeBondingCurvePool.sol, ForgeBondingCurveToken.sol, and BondingCurveTypes.sol) were deployed, tested, and verified. They implement the complete virtual AMM with constant product formula, trading fee collection, and automatic graduation to PlunderSwap V3. The UI was redesigned with an

animated landing page, dual-tab trading interface, real-time price quotes, and progress tracking. The discovery page with unified browsing, the token list with search and filtering, and metadata display with social links were all delivered.

Milestone 4: Final Testing, Documentation, and Public Launch (Month 6, \$10,000)

Comprehensive Foundry test suites cover all contract functionality, including mock contracts for external dependencies. All smart contracts and the frontend application are publicly deployed, accessible, and verified on Sourcify. The codebase includes a README with deployment instructions, environment variable documentation, and usage guides; an AGENTS.md with codebase conventions; and a bonding curve README with mathematical documentation and a parameter tuning guide.

14. Budget Allocation

The \$50,000 grant was allocated as proposed in three categories.

Smart Contract and Backend Development (60%, \$30,000)

This allocation funded the development of all eleven smart contract source files: ForgeTokenFactory, ForgeStandardERC20, ForgeAirdropper, ForgeBondingCurveFactory, ForgeBondingCurvePool, ForgeBondingCurveToken, BondingCurveTypes, ForgeFairLaunchFactory, ForgeFairLaunchPool, FairLaunchTypes, and PlunderInterfaces. It also funded ten deployment and configuration scripts, four comprehensive test suites, the Cloudflare D1 database schema and migration, and the R2 storage configuration.

Frontend and UI Development (25%, \$12,500)

This allocation funded the Next.js application with twelve distinct pages (landing, create token, airdrop, bonding curve launch, bonding curve trading, fair launch creation, fair launch detail, discovery, token list, FAQ, terms, and privacy), eight shared components (app shell, navigation sidebar, top bar, connect wallet button, network selector, page header, theme toggle, and sidebar resize handle), two context providers (Web3 and network), seven ABI definition files, and extensive responsive design with full light and dark mode support.

Ecosystem Integration and Maintenance (15%, \$7,500)

This allocation funded the PlunderSwap V2 and V3 integration code, Torch Wallet connection support, Otterscan block explorer link generation, Sourcify contract verification tooling, network switching between mainnet and testnet, and the Cloudflare Workers deployment pipeline.

15. The ZIL Flywheel

The original proposal described Torchpad as a catalyst for a flywheel effect across the Zilliqa ecosystem. With the implementation complete, here is how that flywheel operates in practice.

Every action on Torchpad generates demand for ZIL. Token creation requires a ZIL fee plus gas. Bonding curve purchases require ZIL as the trading currency. Fair launch contributions denominated in ZIL directly increase demand. Airdrops require gas fees proportional to the number of recipients. And every transaction on Zilliqa's EVM layer consumes gas paid in ZIL.

Torchpad also acts as a permanent liquidity sink. When a bonding curve token graduates, its real ZIL reserves (minus the graduation fee) are locked forever as PlunderSwap V3 liquidity. The LP NFT is burned to an unrecoverable address, removing that ZIL from circulation permanently. Fair launches with indefinite lock durations have the same effect. Even time-limited locks remove ZIL from circulation for months or years.

The result is a self-reinforcing cycle. New tokens launched on Torchpad attract users who need ZIL to participate. Their participation generates trading fees and gas fees, increasing Zilliqa's on-chain activity metrics. Higher activity attracts more developers and projects, who launch more tokens on Torchpad, perpetuating the cycle. The permanent liquidity locks create sustained buy pressure on ZIL, contributing to long-term value appreciation for the entire ecosystem.

16. Risk Assessment and Mitigation

The proposal identified four key risks. Here is how each was addressed.

Smart contract vulnerabilities were mitigated through consistent use of OpenZeppelin's audited library contracts, comprehensive Foundry test suites, systematic application of security best practices (reentrancy guards, input validation, slippage protection, fee refunds), public verification on Sourcify, and an open-source codebase that invites community review.

Low adoption or limited usage was addressed through deep integration with PlunderSwap (ensuring tokens launched on Torchpad have immediate access to existing liquidity), Torch Wallet integration (leveraging an existing user base), a unified discovery interface that aggregates all launches, and support for both mainnet and testnet so developers can experiment risk-free.

Market volatility was addressed by supporting USDC as a fair launch fundraising currency, allowing projects to raise funds in a stablecoin. All grant funds were allocated directly to development as committed.

Scope creep and delays were managed through a clear milestone structure, modular architecture that allowed each system to be developed and tested independently, and established development frameworks (Foundry, Next.js, Cloudflare) to minimize toolchain risk.

17. Success Metrics

The proposal defined several metrics for measuring Torchpad's success. The platform is now instrumented to track all of them.

Tokens launched via Torchpad are tracked through `TokenCreated`, `PoolCreated`, and `FairLaunchCreated` events. The token list page provides a searchable registry of all factory-created tokens, and the discovery page shows all active launches.

Total value of liquidity locked is measurable through the `realZilReserve` in bonding curve pools and the `totalRaised` and `liquidityCurrencyUsed` fields in fair launch pools. Permanent liquidity locks from bonding curve graduations and timed locks from fair launches represent quantifiable locked value.

New on-chain users are tracked through the `contributorCount` field in fair launch pools, which provides a direct measure of unique wallets participating in fundraising events.

Transaction volume is captured through `Buy`, `Sell`, `Contribution`, `Claimed`, and `Refunded` events emitted by the various pool contracts. **On-chain activity growth** can be measured through standard blockchain analytics tools and the Otterscan block explorer by examining cumulative TVL, gas fees, and transaction count over time.

18. Open Source and Licensing

As committed in the proposal, all smart contracts and the frontend application are open source. The complete codebase is publicly available on GitHub at github.com/Ignite-DAO/torchpad. The project is licensed under the Business Source License 1.1 (BSL), which transitions to the MIT license on November 10, 2029. Anyone can read, study, fork, and learn from the codebase today, and after the BSL period ends it becomes fully permissive under MIT.

This licensing approach balances the team's need to protect their investment during the initial growth period with the long-term goal of open, permissionless infrastructure.

No Torchpad-native token has been launched, as stated in the proposal. The platform operates purely as infrastructure, generating revenue through configurable platform fees on token creation, airdrops, bonding curve trading, and fair launch creation.

19. Deployment and Operations

Smart contracts are deployed using Foundry's scripting system. Each contract system has its own deployment script that reads configuration from environment variables with sensible defaults. `DeployFactory.s.sol` deploys the `ForgeTokenFactory`. `DeployAirdropper.s.sol` deploys the `ForgeAirdropper`. `DeployBondingCurveFactory.s.sol` deploys the bonding curve factory with `PlunderSwap` integration addresses and all configurable parameters. `DeployFairLaunchFactory.s.sol` deploys the fair launch factory with the USDC address and `PlunderSwap` router addresses. Post-deployment configuration is handled by dedicated scripts for setting fees and treasury addresses on each contract.

The frontend is deployed to Cloudflare Workers using the OpenNext framework. The pipeline builds the Next.js application, uploads static assets to Cloudflare's edge network, deploys the worker script for server-side rendering and API routes, and configures bindings for R2 and D1. Static assets are served with aggressive caching via the `_headers` configuration file.

All deployment-specific configuration lives in environment variables. Frontend variables use the `NEXT_PUBLIC_` prefix to make them available in the browser, and include contract addresses for both mainnet and testnet, the WalletConnect project ID, and the R2 storage URL. This approach allows the same codebase to serve different environments without code changes.

20. Future Considerations

While Torchpad delivers on all proposed milestones, the platform was designed to be extensible. A formal third-party security audit of all smart contracts is planned. The contracts were designed with auditability in mind, using well-known patterns and libraries to simplify the review process.

A dedicated analytics dashboard showing aggregate platform metrics (total tokens launched, total liquidity locked, trading volumes, user growth over time) is a natural next step. The modular factory pattern makes it straightforward to add new launch mechanisms, such as presales with vesting schedules or Dutch auctions.

As mentioned in the original proposal, integration with ElizaOS for deploying autonomous AI agents is a future direction that would further differentiate Torchpad in the market. Richer project profiles with team information, roadmaps, and community metrics could help users make more informed participation decisions.

21. Conclusion

Torchpad represents the successful delivery of everything proposed in the original grant application. From a permissionless token factory to sophisticated bonding curve mechanics, from structured fair launches to efficient airdrop distribution, the platform provides a complete toolkit for token creation and deployment on Zilliqa 2.0.

The implementation reflects careful attention to security (reentrancy guards, safe transfers, input validation, slippage protection), user experience (guided wizards, real-time feedback, responsive design), and ecosystem integration (PlunderSwap V2/V3, Torch Wallet, Otterscan, Sourcify).

The platform is live, open source at github.com/Ignite-DAO/torchpad, and ready to serve as the foundation for the next wave of token launches on Zilliqa. By lowering the barriers to token creation and distribution, Torchpad is positioned to drive the liquidity, activity, and growth that the Zilliqa ecosystem needs to thrive.

We extend our gratitude to the gZIL Collective for their support and confidence in the Torch Wallet team. We remain committed to evolving Torchpad based on developer needs and community feedback, and to building sustainable, impactful infrastructure for Zilliqa's long-term success.

Appendix A: Contract and Parameter Reference

Deployed Contract Systems

Contract	Networks	Purpose
ForgeTokenFactory	Mainnet & Testnet	Creates standard ERC-20 tokens
ForgeAirdropper	Mainnet & Testnet	Batch token distribution
ForgeBondingCurveFactory	Mainnet & Testnet	Creates bonding curve pools
ForgeFairLaunchFactory	Mainnet & Testnet	Creates fair launch pools

All contracts are verified on Sourcify and can be inspected through the Otterscan block explorer.

Bonding Curve Parameters

Parameter	Default	Range	Effect
Total Supply	1,000,000,000 tokens	Fixed	Every bonding curve token has exactly 1 billion tokens
Graduation Market Cap	8,000,000 ZIL	≥ 1 ZIL	Market cap target for DEX graduation
Initial Virtual ZIL Reserve	22,500 ZIL	> 0	Controls starting price and curve steepness
Trading Fee	100 bps (1%)	0 to 10%	Fee on every buy and sell transaction
Graduation Fee	250 bps (2.5%)	0 to 10%	One-time fee at graduation
V3 Fee Tier	10000 (1%)	V3 tiers	PlunderSwap V3 pool fee for graduated liquidity
Min Buy Amount	0.001 ZIL	Fixed	Minimum ZIL per buy transaction
Min Sell Amount	1e15 tokens	Fixed	Minimum tokens per sell transaction

Fair Launch Parameters

Parameter	Options / Range	Description
Fundraising Currency	ZIL or USDC	The currency contributors send
Tokens for Sale	> 0	Number of tokens distributed to contributors
Soft Cap	> 0	Minimum raise for the sale to succeed
Hard Cap	0 or $\geq$ soft cap	Maximum raise; 0 means unlimited
Max Contribution	$\geq$ 0	Per-wallet limit; 0 means unlimited
Liquidity Percent	51 to 100%	Share of tokens reserved for DEX liquidity
Auto-Listing	On / Off	Automatic PlunderSwap listing on finalize
Router Kind	V2 or V3	PlunderSwap version for listing
V3 Fee Tier	100 / 500 / 2500 / 10000	V3 fee tier if V3 selected
Lock Duration	None / 30d / 90d / 180d / 365d / Indefinite	Liquidity lock period
Whitelist	On / Off + Merkle root	Optional access control

Appendix B: Technology Stack

Layer	Technology	Purpose
Smart Contracts	Solidity 0.8.24 / Foundry	On-chain logic and token management
Contract Libraries	OpenZeppelin 5.x	ERC-20, security, access control
Frontend Framework	Next.js 15 / React 19	Server-rendered web application
Styling	Tailwind CSS v4 / shadcn/ui	Responsive design system
Web3 Integration	Wagmi / RainbowKit / viem	Wallet connection and contract interaction
Form Handling	React Hook Form / Zod	Type-safe form validation
Data Fetching	TanStack React Query	Caching and background refetching
Backend Compute	Cloudflare Workers	Serverless API and SSR
Database	Cloudflare D1 (SQLite)	Token and launch metadata
Object Storage	Cloudflare R2	Project images and logos
Package Manager	Bun	Fast JS runtime and package management
Code Quality	Biome	Linting and formatting
Contract Testing	Foundry (forge test)	Comprehensive test suites with mocks
Contract Verification	Sourcify	Public source code verification

Appendix C: Project File Structure

The following outlines the complete project structure, excluding `node_modules` and build artifacts.

Directory / File	Purpose
<code>contracts/src/ForgeTokenFactory.sol</code>	Token factory contract
<code>contracts/src/ForgeStandardERC20.sol</code>	Standard ERC-20 token
<code>contracts/src/ForgeAirdropper.sol</code>	Batch airdrop contract
<code>contracts/src/bondingcurve/</code>	Bonding curve system (factory, pool, token, types)
<code>contracts/src/fairlaunch/</code>	Fair launch system (factory, pool, types, interfaces)
<code>contracts/test/</code>	Foundry test suites for all contracts
<code>contracts/script/</code>	Deployment and configuration scripts
<code>src/app/</code>	Next.js pages and API routes
<code>src/components/</code>	Shared UI components and shaden/ui elements
<code>src/providers/</code>	Web3 and network context providers
<code>src/lib/</code>	Utility libraries (chains, contracts, formatting)
<code>src/hooks/</code>	Custom React hooks
<code>src/abi/</code>	Contract ABI definitions for frontend interaction
<code>migrations/</code>	D1 database schema
<code>public/</code>	Static assets, logos, and favicons